

# 2026-05-05-p1-f14-sandboxed-shell-execution-design

## Phase 1 / Feature 14 — Sandboxed Shell Execution

**Date:** 2026-05-05 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

---

### 1. Goal

Ship a real, kernel-enforced sandbox for shell commands invoked by the agent. F14 introduces a NEW agent tool registered as `shell_sandboxed` that wraps every command in a hybrid Linux sandbox (bubblewrap when available, native Go `Cloneflags` namespaces otherwise), enforces a default-DENY network policy, applies a CONST-033 deny-list BEFORE the command can reach the sandbox, and fail-closes loudly when the host kernel cannot offer the required isolation primitives.

Three concrete user surfaces ship together:

1. **Agent tool** — `shell_sandboxed` registered via `internal/tools/registry.go::ToolRegistry.registerAllTools()`. The agent invokes it like any other tool; arguments include `command`, `network` (default `false`), `timeout`, and `cwd`.
2. **Slash command** `/sandbox` — `status` (which backend was selected + capability table), `test` (run a deterministic harness command in the sandbox and print exit code + first 256 bytes of stdout), `policy` (print the active deny-list and resource limits).
3. **Config file** — `~/.config/helixcode/sandbox.yaml` (XDG-aware via `$XDG_CONFIG_HOME` with `~/.config` fallback; mode 0600; parent dir 0700). The user can extend the deny-list, set memory/cpu caps, allow specific outbound hosts, and override defaults. A missing file uses built-in defaults (no error).

Scope is **Linux v1**. macOS Seatbelt and Windows Job Object are explicitly deferred to F14.5 (§8). On non-Linux hosts the manager fails closed with a clear message — no fallback “noop” sandbox is offered, ever.

The existing `helix_code/internal/tools/shell/` package (which ships its own `Command / ShellExecutor / Sandbox` types and has pre-existing build failures predating F09) is **left untouched**. F14 is a parallel, additive package at `helix_code/internal/tools/sandbox/` with its own types and its own tool. There is intentionally no shared `SandboxConfig` symbol between the two packages — name reuse across packages is fine in Go and avoiding it would mean either fixing pre-existing breakage or contorting F14’s API; both are out of scope.

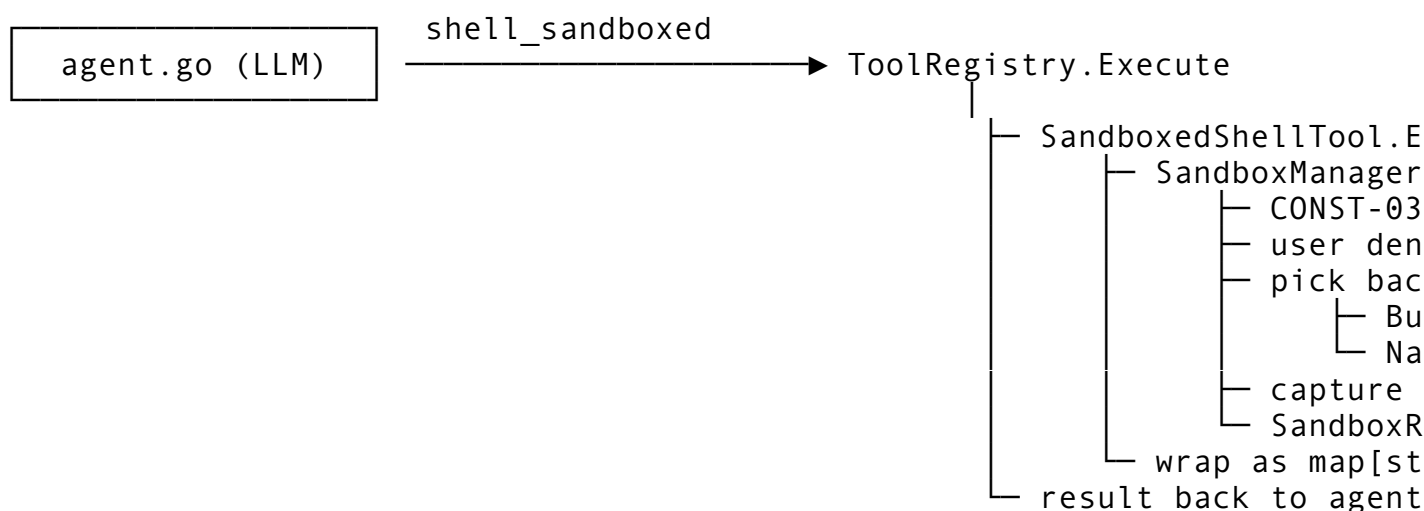
The scope of F14 is **kernel-enforced isolation of a single shell command per invocation**. Long-running daemons inside the sandbox, persistent named sandbox sessions, in-sandbox interactive TTYs, and seccomp-bpf custom filters are explicitly deferred (§8).

## 2. Architecture

Five layers, each with a single responsibility, all under `internal/tools/sandbox/`:

- `SandboxDetector` — runs once at startup. Probes which `bwrap`, `cat /proc/sys/kernel/unprivileged_users_nice`, the presence of `/sys/fs/cgroup/cgroup.controllers` (cgroups v2 unified hierarchy), `runtime.GOOS`, and the running user's UID. Emits a `SandboxCapabilities` struct.
- `SandboxManager` — holds the chosen `SandboxBackend` plus the loaded `SandboxConfig`. Its `Execute(ctx, *SandboxRequest)` is the only public path. Enforces (a) CONST-033 deny-list, (b) user deny-list, (c) network gating, (d) timeout, (e) backend dispatch. Fail-closes when no backend is viable.
- `SandboxBackend interface` — two implementations:
  - `BubblewrapBackend` — wraps every request as a `bwrap ...` subprocess invocation. `Argv` built deterministically from `SandboxPolicy`.
  - `NativeBackend` — uses `syscall.SysProcAttr.Cloneflags` to spawn the target with `CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWUTS | CLONE_NEWIPC | CLONE_NEWUSER`, plus `CLONE_NEWNET` when network is denied. Mounts `/proc` inside the new mount namespace via a tiny re-exec helper. Best-effort cgroups v2 memory cap when writable.
- `SandboxedShellTool` — implements the existing `Tool` interface (`Name/Description/Schema/Category/Validate/Execute`) and delegates to the manager. Tool name is the literal string `shell_sandboxed`. Category `CategoryShell` (reused from `registry.go`).
- `SandboxConfigLoader` — loads `~/.config/helixcode/sandbox.yaml` via `gopkg.in/yaml.v3` (already in `go.mod`). Parent dir created with mode `0700`, file written with `os.OpenFile(0_CREATE|0_WRONLY, 0600)` (mirrors `F12 wizard_writer` pattern). Missing file → built-in defaults, no error.

The manager and the slash command both consume `SandboxCapabilities` directly so `/sandbox status` shows the same values the manager used at backend-selection time.



## 3. Components

### 3.1 New files

- `helix_code/internal/tools/sandbox/types.go` — `SandboxConfig`, `SandboxPolicy`, `SandboxCapabilities`, `SandboxRequest`, `SandboxResult`, `SandboxBackend` interface, `SandboxBackendKind` enum, `deny-list` constants. No external deps.
- `helix_code/internal/tools/sandbox/types_test.go`.
- `helix_code/internal/tools/sandbox/detector.go` — `Detect()` `SandboxCapabilities` (probes `bwrap`, `userns`, `cgroups v2`, `GOOS`).
- `helix_code/internal/tools/sandbox/detector_test.go` — uses an injected `Probe` interface so unit tests don't depend on the host kernel.
- `helix_code/internal/tools/sandbox/bubblewrap_backend.go` — `argv` builder + `Run` via `os/exec.CommandContext`.
- `helix_code/internal/tools/sandbox/bubblewrap_backend_test.go` — `argv` assertions against a fake `SubprocessRunner`.
- `helix_code/internal/tools/sandbox/native_backend.go` — `SysProcAttr.Cloneflags` setup + `Run`.
- `helix_code/internal/tools/sandbox/native_backend_test.go` — gated unit tests when `unprivileged_userns_clone == 1`.
- `helix_code/internal/tools/sandbox/manager.go` — backend selection + policy enforcement.
- `helix_code/internal/tools/sandbox/manager_test.go`.
- `helix_code/internal/tools/sandbox/sandboxed_shell_tool.go` — `Tool` interface implementation; tool name `shell_sandboxed`.
- `helix_code/internal/tools/sandbox/sandboxed_shell_tool_test.go`.
- `helix_code/internal/tools/sandbox/config_loader.go` — `YAML` loader + `secret-safe` writer (mirrors `F12 wizard_writer.go`).
- `helix_code/internal/tools/sandbox/config_loader_test.go`.
- `helix_code/internal/commands/sandbox_command.go` — `/sandbox` slash command (`status / test / policy`).
- `helix_code/internal/commands/sandbox_command_test.go`.
- `helix_code/tests/integration/sandbox_test.go` — `//go:build integration`, gated per §5.
- `helix_code/tests/integration/cmd/p1f14_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f14-sandboxed-shell-execution/CHALLENGE.md` + `run.sh`.

### 3.2 Modified files

- `helix_code/internal/tools/registry.go` — register `SandboxedShellTool` in `registerAllTools()`. Add `SetSandboxManager(*sandbox.SandboxManager)` in the same shape as the existing `F13 SetLSPManager`. The tool is registered lazily from `SetSandboxManager` (so unit tests of the registry that don't supply a manager don't see `shell_sandboxed`). NO changes to `Execute` semantics — `shell_sandboxed` is just another tool, no auto-trigger needed.
- `helix_code/cmd/cli/main.go` — call `sandbox.Detect()`, load `~/.config/helixcode/sandbox.yaml`, construct a `SandboxManager`, call

registry.SetSandboxManager(...), register /sandbox slash command. **No cobra subcommand** (Q5=C — keeps surface light).

### 3.3 Types

```
// internal/tools/sandbox/types.go

type SandboxBackendKind int
const (
    BackendNone          SandboxBackendKind = 0
    BackendBubblewrap    SandboxBackendKind = 1
    BackendNative        SandboxBackendKind = 2
)
func (k SandboxBackendKind) String() string

// SandboxCapabilities is what the detector found at startup.
// Read-only
// after Detect() returns.
type SandboxCapabilities struct {
    GOOS                string // runtime.GOOS
    BubblewrapPath       string // "" if not on PATH
    UnprivilegedUsersns bool   // /proc/sys/kernel/
                        unprivileged_usersns_clone == 1
    CgroupsV2Mounted    bool   // /sys/fs/cgroup/
                        cgroup.controllers exists
    UID                  int    // os.Getuid()
    SelectedBackend      SandboxBackendKind
    SelectionReason       string // human-readable why we
                        picked it
}

// SandboxPolicy is the per-invocation isolation policy. Built
// from
// SandboxConfig + per-call request overrides.
type SandboxPolicy struct {
    AllowNetwork    bool // default false
    ReadOnlyPaths  []string // bind-mounted ro
    ReadWritePaths []string // bind-mounted rw (default: cwd only)
    Cwd             string // working directory inside
                        sandbox
    MaxMemoryMB     int // 0 = no cgroup memory cap
    MaxCPUSeconds   int // 0 = no rlimit cpu cap
    Timeout         time.Duration // ctx-enforced wallclock cap
                        (default 30s, hard ceiling 10m)
    Env             []string // KEY=VAL; empty → minimal
                        default
    DenyList        []string // additional deny-list
                        (CONST-033 always applied)
```

```

}

// SandboxRequest is what SandboxedShellTool builds from agent
// params.
type SandboxRequest struct {
    Command
        string // exact shell snippet (run via /bin/sh -c)
    Policy  SandboxPolicy
}

type SandboxResult struct {
    ExitCode  int
    Stdout    []byte
    Stderr    []byte
    Backend   SandboxBackendKind
    Duration  time.Duration
    Truncated bool // true if stdout/stderr were
        truncated to MaxOutputBytes
}

type SandboxBackend interface {
    Kind() SandboxBackendKind
    Run(ctx context.Context, req *SandboxRequest)
        (*SandboxResult, error)
}

// SandboxConfig is the persisted config (YAML). Loaded once at
// startup
// from ~/.config/helixcode/sandbox.yaml.
type SandboxConfig struct {
    DefaultPolicy SandboxPolicy `yaml:"default_policy"`
    DenyList      []string `yaml:"deny_list" //
        user-extended; merged with CONST-033 list
    AllowedHosts []string `yaml:"allowed_hosts" //
        reserved (network=true gating); v1: informational
    MaxOutputBytes int64 `yaml:"max_output_bytes" //
        default 10*1024*1024
}

// CONST-033 deny-list – always applied, never overridable.
// Matched against
// the parsed argv head AND scanned as substring with word-
// boundary regex
// on the raw command string. Both checks must pass.
var ConstitutionalDenyList = []string{
    "systemctl suspend", "systemctl hibernate", "systemctl
        hybrid-sleep",

```

```

        "systemctl suspend-then-hibernate", "systemctl poweroff",
        "systemctl halt",
        "systemctl reboot", "systemctl kexec",
        "shutdown", "halt", "poweroff", "reboot", "kexec",
        "pm-suspend", "pm-hibernate", "pm-suspend-hybrid",
        "loginctl suspend", "loginctl hibernate", "loginctl hybrid-
        sleep",
        "loginctl poweroff", "loginctl reboot",
    }
}

// internal/tools/sandbox/manager.go

type SandboxManagerOptions struct {
    Capabilities SandboxCapabilities
    Config        SandboxConfig
    Backend
        SandboxBackend          // injectable for unit tests
    Now            func()
        time.Time                // injectable; default time.Now
    Logger          *log.Logger
}

type SandboxManager struct {
    opts SandboxManagerOptions
    mu    sync.Mutex
}

func NewSandboxManager(opts SandboxManagerOptions)
    *SandboxManager
func (m *SandboxManager) Execute(ctx context.Context, req
    *SandboxRequest) (*SandboxResult, error)
func (m *SandboxManager) Capabilities() SandboxCapabilities
func (m *SandboxManager) ActivePolicy() SandboxPolicy
func (m *SandboxManager) DenyList()
    []string                // CONST-033 + user
    deny-list, deduped
func (m *SandboxManager) Close(ctx context.Context)
    error                // no-op in v1; reserved

// internal/tools/sandbox/sandboxed_shell_tool.go

type SandboxedShellTool struct {
    manager *SandboxManager
}

func NewSandboxedShellTool(m *SandboxManager) *SandboxedShellTool
func (t *SandboxedShellTool) Name() string { return
    "shell_sandboxed" }

```

```

func (t *SandboxedShellTool) Description() string { return
    "..."}
func (t *SandboxedShellTool) Schema() ToolSchema // command
    (string, required), network (bool, default false),
    timeout_seconds (int, default 30), cwd (string, optional)
func (t *SandboxedShellTool) Category() ToolCategory //
    CategoryShell
func (t *SandboxedShellTool) Validate(params
    map[string]interface{}) error
func (t *SandboxedShellTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error)

```

The agent tool result is a `map[string]interface{}` with keys `exit_code`, `stdout`, `stderr`, `backend`, `duration_ms`, `truncated`. Stdout and stderr are strings (UTF-8; invalid sequences replaced) capped at `MaxOutputBytes` total.

### 3.4 User surfaces

**Agent tool call** (registered with `ToolRegistry`):

| Tool name       | Schema                                                                                         | Returns                                                               |
|-----------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| shell_sandboxed | {command: string,<br>network?:<br>bool=false,<br>timeout_seconds?:<br>int=30, cwd?:<br>string} | {exit_code, stdout,<br>stderr, backend,<br>duration_ms,<br>truncated} |

**Slash command** `/sandbox`: `- /sandbox status` — table: GOOS, bubblewrap path (or “not installed”), unprivileged users yes/no, cgroups v2 yes/no, selected backend, selection reason. `- /sandbox test` — runs `/bin/echo p1f14-sandbox-ok` inside the sandbox with `network=false`; prints exit code, captured stdout, backend used. Acts as a one-line live smoke from the user’s prompt. `- /sandbox policy` — prints the active `SandboxPolicy` (default policy + merged deny-list). The `CONST-033` entries are clearly marked with a `[CONST-033]` prefix so the user can see they cannot be removed.

**No cobra subcommand** (per Q5=C). The slash command is sufficient for inspection; agent invocations go through the tool.

### 3.5 Existing-code constraints

`helix_code/internal/tools/shell/` already defines `SandboxConfig`, `SandboxBackend`, `Sandbox`, `Command`, `ExecutionResult`, `NetworkMode`, etc. **F14 deliberately does NOT import or modify that package.** Names in the new `sandbox` package are independent (Go scopes types per package). The pre-existing build failures in `internal/tools/shell/` predate F09 and are out of scope; the `shell_sandboxed` tool is implemented directly via `os/exec.CommandContext` plus the new backend, without the broken `ShellExecutor`. The existing `bash` tool stays as-is — F14 does not modify it (Q4=B).

## 4. Data flow

### 4.1 Startup wiring (cmd/cli/main.go)

```
NewCLI()

- └ caps := sandbox.Detect()
- └ cfg, err := sandbox.LoadConfig("") // "" → ~/.config/hel
- └ if err != nil: log WARN, cfg = DefaultSandboxConfig()
- └ backend, selectErr := sandbox.SelectBackend(caps) // bwrap > native
- └ if selectErr != nil: log INFO once with the install hint; backend = n
- └ mgr := sandbox.NewSandboxManager(SandboxManagerOptions{
  - Capabilities: caps, Config: cfg, Backend: backend,)
- └ registry.SetSandboxManager(mgr) // also registers shell_sandbo
- └ slashRegistry.Register(commands.NewSandboxCommand(mgr))
- └ defer mgr.Close(ctx)

```

If `backend == nil`, `SetSandboxManager` is still called — the tool is registered but `Execute` returns the documented fail-closed error so the agent gets a clear message rather than a silent unsandboxed run.

### 4.2 Per-command Execute flow

```
SandboxedShellTool.Execute(ctx, params)

- └ req := buildRequest(params, mgr.ActivePolicy())
  - └ apply per-call overrides (network, timeout_seconds, cwd)
  - └ inherit DenyList, ReadOnly/ReadWrite paths, MaxMemoryMB, MaxCPU
- └ res, err := mgr.Execute(ctx, req)
- └ return map{exit_code, stdout, stderr, backend, duration_ms, truncated

```

```
SandboxManager.Execute(ctx, req)

- └ if mgr.opts.Backend == nil:
  - └ return nil, ErrSandboxUnavailable // see §5.1 for exact text
- └ if matchesAny(req.Command, ConstitutionalDenyList):
  - └ return nil, ErrConstitutionalDeny // CONST-033; rejected BEFORE
- └ if matchesAny(req.Command, mgr.opts.Config.DenyList):
  - └ return nil, ErrPolicyDeny
- └ ctx, cancel := context.WithTimeout(ctx, req.Policy.Timeout)
- └ defer cancel()
- └ res, err := mgr.opts.Backend.Run(ctx, req)
- └ return res, err

```

`matchesAny` runs each deny-list entry through both (a) tokenised argv-head match against the parsed command (shellwords style, no eval), and (b) word-boundary regex on the raw command. Both share the same denylist. This catches `systemctl suspend` (whitespace), `systemctl suspend; ls` (chained), `bash -c 'systemctl suspend'` (nested shell).

### 4.3 Bubblewrap argv construction

```
bwrap

- die-with-parent
- new-session
- unshare-pid --unshare-ipc --unshare-uts --unshare-cgroup

```



```

--unshare-net                # only when AllowNetwork=
--proc /proc
--dev /dev
--tmpfs /tmp
--ro-bind / /                # default ro filesystem v
--bind <cwd> <cwd>           # writable: only the cwd
--chdir <cwd>
--setenv PATH /usr/local/bin:/usr/bin:/bin
--                            # end of bwrap args
/bin/sh -c <command>

```

ReadOnlyPaths and ReadWritePaths from the policy add `--ro-bind / --bind` pairs. Memory cap is best-effort cgroups v2 via `--cgroup` when the unified hierarchy is writable; otherwise we rely on per-process rlimits set via `cmd.SysProcAttr` after the bwrap process forks (rlimits inherit). Argv is built deterministically, sorted, and unit-tested without invoking bwrap.

## 4.4 Native backend cloneflags

```

cmd.SysProcAttr = &syscall.SysProcAttr{
    Cloneflags:  syscall.CLONE_NEWPID | syscall.CLONE_NEWNS |
                 syscall.CLONE_NEWUTS |
                 syscall.CLONE_NEWIPC | syscall.CLONE_NEWUSER,
    UidMappings: []syscall.SysProcIDMap{{ContainerID: 0,
        HostID: os.Getuid(), Size: 1}},
    GidMappings: []syscall.SysProcIDMap{{ContainerID: 0,
        HostID: os.Getgid(), Size: 1}},
    Pdeathsig:   syscall.SIGKILL,
}
if !req.Policy.AllowNetwork {
    cmd.SysProcAttr.Cloneflags |= syscall.CLONE_NEWNET
}

```

A tiny re-exec helper inside the new mount namespace remounts `/proc`, then execves `/bin/sh -c <command>`. The helper is a separate small binary built from `internal/tools/sandbox/cmd/native_helper/main.go` and located by absolute path captured at startup via `os.Executable() + sibling lookup`; if absent, the native backend fail-closes.

Resource caps: `cmd.SysProcAttr` carries `Rlimit` for `RLIMIT_AS` (memory) and `RLIMIT_CPU`. Cgroups v2 memory cap is best-effort via writing `memory.max` under a per-invocation `helixcode-sandbox-<pid> slice` when `/sys/fs/cgroup/cgroup.subtree_control` is writable; otherwise rlimits are the fallback.

## 4.5 Network-deny enforcement

**Default DENY** (Q3=A). The policy sets `AllowNetwork=false` unless the agent explicitly passes `network: true` in the tool params. With `false`: - bubblewrap: `--unshare-net` adds an empty network namespace (no interfaces, no DNS). - native: `CLONE_NEWNET` does the same.

A test phase verifies that `curl https://example.com` fails with a connection / DNS error in the default-deny mode, and succeeds (HTTP 2xx) when `network: true` is passed. Both checks gated on a working backend.

## 4.6 CONST-033 deny-list flow

The deny-list is checked **before** any subprocess is spawned. The check is in `SandboxManager.Execute`, not in the backend. Both backends therefore inherit it for free. The check is non-overridable: `SandboxConfig.DenyList` may only ADD to it, never subtract. `/sandbox policy` shows the merged list with [CONST-033] markers.

## 5. Error handling, edge cases, and anti-bluff

### 5.1 Error paths

- **No backend available** (no bwrap + no userns + non-Linux) — `SandboxManager.Execute` returns `ErrSandboxUnavailable` whose `.Error()` is exactly:  
  
`sandboxing unavailable: install bubblewrap (apt install bubblewrap; dn`  
  
The agent tool surfaces this verbatim. **No silent unsandboxed run, ever.**
- **Non-Linux host** — `runtime.GOOS != "linux"` short-circuits backend selection to nil; same `ErrSandboxUnavailable` with platform note.
- **Constitutional deny-list hit** — `ErrConstitutionalDeny: "command rejected: <token> matches CONST-033 power-management ban"`. Returns BEFORE sandbox spawn.
- **User deny-list hit** — `ErrPolicyDeny: "command rejected: <token> matches sandbox.yaml deny_list"`.
- **Timeout** — `context.DeadlineExceeded` is wrapped into a `SandboxResult{ExitCode: -1, Stderr: "sandbox timeout after Xs", Truncated: false}`; tool returns the result (NOT err) so the agent sees the timeout cleanly.
- **OOM** — backend reports exit signal SIGKILL from cgroups OOM killer (or RLIMIT\_AS failure); `SandboxResult.ExitCode = -9` and `Stderr` carries `"sandbox out-of-memory: hit MaxMemoryMB=<n>"`.
- **bwrap missing user-mapping caps** — bwrap exits with a specific diagnostic; manager surfaces `bubblewrap setup failed: <stderr>` and the user is pointed at `unprivileged-usersns + setuid bwrap docs`.
- **Concurrent invocations** — manager is safe for concurrent `Execute` calls; no shared mutable state in the per-call request path. Each invocation gets its own backend `Run` goroutine and its own subprocess.
- **Stdout/stderr exceeds MaxOutputBytes** — captured to `byte-bounded bytes.Buffer`, `Truncated=true`, the tail beyond the cap is discarded.

- **Manager Close while an Execute is in flight** — Close is a no-op in v1; in-flight calls finish under their own ctx. Reserved for future cgroup cleanup work.

## 5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F14 is “sandbox test passed” with no real bubblewrap or users available, and the test silently degrading to an unsandboxed os/exec.Run. The gating policy below is non-negotiable.

1. **Detector accuracy is the load-bearing primitive.** The Detect() function MUST be honest. It probes:
  - which bwrap via exec.LookPath("bwrap") — non-empty path means bubblewrap usable.
  - cat /proc/sys/kernel/unprivileged\_users\_clone — value 1 means unprivileged users is enabled. Older kernels (pre 5.10) without this sysctl and with users enabled by default count as enabled.
  - stat /sys/fs/cgroup/cgroup.controllers — file exists means cgroups v2 unified hierarchy is mounted.
  - runtime.GOOS — non-linux short-circuits to no-backend.
  - os.Getuid() — captured for diagnostics; unprivileged users is the primary gate. The detector emits SandboxCapabilities with **no defaults guessed** — every field reflects an actual probe, and SelectionReason is a one-line human-readable string captured by the manager so /sandbox status cannot lie.
2. **Backend selection** in SelectBackend(caps SandboxCapabilities) (SandboxBackend, error):
  - bubblewrap path non-empty → BubblewrapBackend{Path: caps.BubblewrapPath, reason "bubblewrap (preferred)".
  - else: caps.UnprivilegedUsers == true AND caps.GOOS == "linux" → NativeBackend{...}, reason "native users fallback (no bwrap)".
  - else: returns (nil, ErrSandboxUnavailable). **Fail closed.** Manager records the capability gap in its diagnostic state for /sandbox status.
3. **Unit tests** — mocks OK ONLY at the SubprocessRunner boundary (an interface with Run(ctx, \*exec.Cmd) ([]byte, []byte, int, error)). This lets us validate command-line construction (full bwrap argv, native cloneflags + uid/gid maps) without spawning real subprocesses. NO mocks of the manager. NO mocks of policy enforcement — those are tested by direct calls to SandboxManager.Execute with a fake backend that returns a canned SandboxResult.
4. **Integration tests** (-tags=integration) — gated on real kernel features:
  - TestSandbox\_Bubblewrap\_RunsEcho — gated on exec.LookPath("bwrap") == nil. When absent: t.Skip("SKIP-OK: P1-F14 bubblewrap not on PATH (install: apt install bubblewrap OR dnf install bubblewrap)").
  - TestSandbox\_Bubblewrap\_NetworkDeniedByDefault — same gate. Asserts curl --max-time 2 https://example.com exits non-zero.
  - TestSandbox\_Bubblewrap\_NetworkAllowedWhenOptedIn — same gate; gated additionally on outbound network to a controlled host (skip if offline).

- `TestSandbox_Native_RunsEcho` — gated on `unprivileged_usersns_clone == 1`; on disabled: `t.Skip("SKIP-OK: P1-F14 unprivileged user namespaces disabled (echo 1 > /proc/sys/kernel/unprivileged_usersns_clone)")`.
- `TestSandbox_Native_NetworkDeniedByDefault` — same gate.
- `TestSandbox_FailsClosedWhenNoBackend` — runs ALWAYS; constructs a manager with a synthetic `SandboxCapabilities{}` (zero values), asserts `ErrSandboxUnavailable` with the exact documented message. **This test proves fail-closed semantics regardless of host capabilities.**
- `TestSandbox_RejectsConstitutionalDenyList` — runs ALWAYS; calls `Execute` with command: `"systemctl suspend"`, asserts `ErrConstitutionalDeny` and that NO subprocess was spawned (uses fake backend with a counter).

SKIP-OK: is the canonical marker required by `make no-silent-skips`. Bare `t.Skip()` is forbidden.

5. **Challenge harness** — exercises three phases, two gated:

- **Phase A — detector + fail-closed (always runs)** — probes capabilities, prints them, builds a manager with a forced empty `SandboxCapabilities`, calls `Execute("/bin/echo hi")`, asserts `ErrSandboxUnavailable` with documented text. Then asserts `Execute("systemctl suspend")` returns `ErrConstitutionalDeny` even when a working backend IS available. ~6 [PASS] lines.
- **Phase B — bubblewrap (gated)** — [skipped: `bubblewrap` not on PATH (`apt install bubblewrap`)] if absent. Otherwise: 4 [PASS] lines (runs `echo`, network denied, network allowed, deny-list rejected before spawn).
- **Phase C — native (gated)** — [skipped: `unprivileged_usersns disabled (echo 1 > /proc/sys/kernel/unprivileged_usersns_clone)`] if absent. Otherwise: 3 [PASS] lines (runs `echo`, network denied, deny-list rejected). Final summary: DETECTOR=6/6 PASS; BWRAP=<n>/4 PASS; NATIVE=<n>/3 PASS.

6. **Anti-bluff rule for the Challenge:** the harness MUST present three clearly separated sections. A reader of the output cannot confuse “fail-closed semantics work” with “bubblewrap actually sandboxed a real command.” The Challenge MUST exit non-zero on any assertion failure within phases that did run, and explicitly note skipped phases.

7. **Concrete forbidden phrases** (anti-bluff smoke clean check, applied to every new file):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO
implement\|placeholder" \
internal/tools/sandbox internal/commands/
sandbox_command.go && echo BLUFF || echo clean
```

Must always print `clean`.

8. **No silent unsandboxed fallback.** Under no codepath does F14 fall back to running a command outside the sandbox. The only paths are: real sandbox, deny-list rejection, or `ErrSandboxUnavailable`. If we ever change this, it requires a constitutional amendment because the entire user-trust model around `shell_sandboxed` rests on this.

## 6. Testing

### 6.1 Unit (mocks OK, SubprocessRunner boundary only)

- `TestSandboxCapabilities_AllFieldsPopulatedFromProbes.`
- `TestSelectBackend_PrefersBubblewrap_WhenPresent.`
- `TestSelectBackend_FallsBackToNative_WhenNoBwrapButUserns.`
- `TestSelectBackend_FailsClosed_WhenNeither.`
- `TestSelectBackend_FailsClosed_WhenNonLinux` (force caps.GOOB = "darwin").
- `TestBubblewrapBackend_ArgvShape_DefaultDenyNetwork` — asserts presence of `--unshare-net`.
- `TestBubblewrapBackend_ArgvShape_NetworkAllowed` — asserts absence of `--unshare-net` when policy allows.
- `TestBubblewrapBackend_ArgvShape_BindsCwdRW.`
- `TestBubblewrapBackend_ArgvShape_RootBindIsReadOnly.`
- `TestNativeBackend_Cloneflags_IncludeUserns` — asserts the `Cloneflags` bitmap.
- `TestNativeBackend_Cloneflags_AddsNewnetWhenNetDenied.`
- `TestSandboxManager_RejectsConstitutionalDenyList_BeforeSpawn` — counter on fake backend stays at 0.
- `TestSandboxManager_RejectsUserDenyList_BeforeSpawn.`
- `TestSandboxManager_AppliesTimeout.`
- `TestSandboxManager_TruncatesOutputAtMaxBytes.`
- `TestSandboxManager_FailsClosed_WhenBackendNil.`
- `TestSandboxedShellTool_BuildsRequestFromParams.`
- `TestSandboxedShellTool_NetworkDefaultsFalse.`
- `TestSandboxedShellTool_ResultMapShape.`
- `TestSandboxConfigLoader_LoadsYAML.`
- `TestSandboxConfigLoader_MissingFile_ReturnsDefaults_NoError.`
- `TestSandboxConfigLoader_WriterEnforcesMode0600.`
- `TestSandboxCommand_Status_RendersTable.`
- `TestSandboxCommand_Test_RunsEchoAndPrintsExit.`
- `TestSandboxCommand_Policy_PrintsConstitutionalEntriesWithPrefix.`

### 6.2 Integration (-tags=integration, gated per §5.2)

- `TestSandbox_Bubblewrap_RunsEcho` — SKIP-OK: P1-F14 bubblewrap not on PATH (install: `apt install bubblewrap` OR `dnf install bubblewrap`).
- `TestSandbox_Bubblewrap_NetworkDeniedByDefault` — same skip.
- `TestSandbox_Bubblewrap_NetworkAllowedWhenOptedIn` — same skip; additional offline-skip with SKIP-OK: P1-F14 host has no outbound HTTPS connectivity.
- `TestSandbox_Native_RunsEcho` — SKIP-OK: P1-F14 unprivileged user namespaces disabled (`echo 1 > /proc/sys/kernel/unprivileged_userns_clone`).
- `TestSandbox_Native_NetworkDeniedByDefault` — same skip.
- `TestSandbox_FailsClosedWhenNoBackend` — never skipped.
- `TestSandbox_RejectsConstitutionalDenyList` — never skipped.

## 6.3 Challenge

- **Phase A — detector + fail-closed (always runs)** — 6 [PASS] lines including the CONST-033 rejection check that runs even when a backend is available (so the deny-list path is exercised on hosts that have bwrap).
- **Phase B — bubblewrap (gated)** — 4 [PASS] lines or [skipped: bubblewrap not on PATH (install: apt install bubblewrap)].
- **Phase C — native (gated)** — 3 [PASS] lines or [skipped: unprivileged users disabled (echo 1 > /proc/sys/kernel/unprivileged\_users\_max)].
- Final summary line: DETECTOR=6/6 PASS; BWRAP=<n>/4 PASS; NATIVE=<n>/3 PASS.
- The Challenge MUST exit non-zero on any assertion failure within phases that did run.

## 7. Cross-platform

Linux only for v1. Concretely: - `runtime.GOOS != "linux"` → `SelectBackend` returns (`nil`, `ErrSandboxUnavailable`) with the message: "sandboxing unavailable: F14 v1 supports Linux only (macOS Seatbelt + Windows Job Object deferred to F14.5)". - The manager always loads on every platform (so `/sandbox` status works everywhere and reports the gap). - `shell_sandboxed` tool is registered on every platform (so the agent sees the tool and gets the documented error rather than a “tool not found” surprise). - Build tags are NOT used on the public API (`types.go`, `manager.go`, `sandboxed_shell_tool.go`, `config_loader.go`, `detector.go` all build on every GOOS). Only `native_backend.go` carries `//go:build linux`; on non-Linux it has a stub `native_backend_other.go` with `//go:build !linux` that returns the platform error from `Run`.

## 8. Out of scope (deferred)

- **macOS Seatbelt sandbox profile** — F14.5 (separate spec). Will use `sandbox-exec` and a generated `.sb` policy file.
- **Windows Job Object + ACL sandbox** — F14.5. Job Object via `kernel32.dll` syscalls; AppContainer integration optional.
- **seccomp-bpf custom filter** — F14.5. v1 uses bubblewrap’s default seccomp policy (which already blocks the most dangerous syscalls); custom filters require either `libseccomp` cgo bindings or hand-rolled BPF — both out of scope for v1.
- **IO bandwidth quotas** — v1 has memory + CPU only. cgroups v2 `io.max` requires writable cgroup hierarchy and is host-config-dependent.
- **Persistent named sandbox sessions** — v1 is one-shot per `Execute` call. Long-running sandboxed shells / interactive TTYs are deferred.
- **Custom seccomp policy upload via `sandbox.yaml`** — v1 only allows deny-list extension; full seccomp programs deferred.
- **Privileged-mode escape hatch** — explicitly NOT supported. There is no flag to disable the sandbox; users who want unsandboxed shell run the existing `bash` tool.
- **Non-Linux fallback to a “noop” sandbox** — explicitly NOT supported. Per §7, non-Linux fail-closes.

## 9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has THREE sections (detector/fail-closed, bwrap, native), the first always runs against real subprocesses + the real fail-closed path, the latter

two are explicitly gated and never claim PASS without a runtime call. [skipped: ...] lines name the missing capability AND the install command. Detector accuracy is itself a tested invariant.

- **CONST-033 (Host Power Management ban)** — `ConstitutionalDenyList` in `types.go` enumerates `systemctl suspend|hibernate|hybrid-sleep|suspend-then-hibernate|poweroff|halt|reboot|kexec, bare shutdown|halt|poweroff|reboot|kexec, pm-suspend|pm-hibernate|pm-suspend-hybrid`, and the `loginctl` equivalents. The check runs in `SandboxManager`. Execute BEFORE any subprocess is spawned (asserted by `TestSandboxManager_RejectsConstitutionalDenyList_BeforeSpawn` with a spawn-counter on the fake backend). Matching is tokenised + word-boundary regex — both pass to handle whitespace, chained commands, and nested `bash -c '...'`.
- **CONST-039** — F14 ships with a Challenge in `challenges/p1-f14-sandboxed-shell-execution/` and an evidence harness at `tests/integration/cmd/plf14_challenge/main.go`.
- **CONST-042 (No-Secret-Leak)** — `~/ .config/helixcode/sandbox.yaml` is written via `os.OpenFile(path, O_CREATE|O_WRONLY|O_EXCL, 0o600)` with parent dir created at `0o700`. The Challenge verifies file mode ( $\leq 0600$ ) and parent dir mode ( $\leq 0700$ ). Loader rejects files with broader perms with `ErrInsecurePerms`.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force.
- **No-Mocks-In-Production (Universal Rule 2)** — `SandboxManager`, both backends, and the agent tool are real, talking to real subprocesses. Mocks live only in `_test.go` files at the `SubprocessRunner` boundary. The fake `SandboxBackend` used in deny-list tests is a unit-test artifact only and is never compiled into the production binary.

## 10. Open questions resolved

| Q                          | Answer             | Resolution                                                                                                                                                                                                        |
|----------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q1: platform scope         | (B) Linux first    | <code>cgroups v2</code> + <code>PID/mount/network/user namespaces</code> ; macOS Seatbelt + Windows Job Object deferred to F14.5                                                                                  |
| Q2: backend implementation | (C) hybrid         | prefer bubblewrap ( <code>bwrap</code> ) when on <code>PATH</code> ; else native Go via <code>syscall.SysProcAttr.Cloneflags</code> ; both detected at startup; backend reported via <code>/sandbox status</code> |
| Q3: network policy         | (A) default DENY   | sandboxed commands have no network unless agent passes <code>network: true</code> per call; both backends enforce via <code>NEWNET / --unshare-net</code>                                                         |
| Q4: tool integration       | (B) NEW tool       | new <code>SandboxedShellTool</code> registered as <code>shell_sandboxed</code> ; existing <code>internal/tools/shell/</code> left untouched (pre-existing build failures predate F09 and are out of scope)        |
| Q5: user surface           | (C) config + slash | primary config at <code>~/ .config/helixcode/sandbox.yaml</code> (XDG-aware, mode <code>0600</code> ); <code>/sandbox slash</code> with <code>status/test/policy</code> ; <b>no cobra subcommand</b>              |